

Chapter Overview

Chapter 12 covers internationalisation (i18n) and localisation (l10n) in ShopVerse: Spring MessageSource, locale resolution, ResourceBundle, number/currency/date formatting, Thymeleaf i18n, REST API localisation, RTL language support, Unicode handling, database multilingual content, and testing strategies.

Sub-Chapter	Topic
12-01	i18n vs l10n – concepts, Locale, IETF language tags
12-02	Spring MessageSource – ReloadableResourceBundleMessageSource
12-03	Locale Resolution – Accept-Language, cookie, param, path strategies
12-04	Message Properties – key naming conventions, pluralisation
12-05	Number, Currency & Date Formatting – NumberFormat, DateTimeFormatter
12-06	REST API i18n – localised error messages, response fields
12-07	Thymeleaf i18n – th:text #{...}, fragment localisation
12-08	Database Multilingual Content – translation tables pattern
12-09	RTL Language Support – Arabic, Hebrew layout considerations
12-10	Unicode Handling – UTF-8 everywhere, collation, normalisation
12-11	ICU4J – complex pluralisation, number spellout, CLDR rules
12-12	Testing i18n – parameterised locale tests

Term	Full Name	Meaning	Example
i18n	Internationalisation	Design software to support multiple locales without code changes	Extract strings to code properties files; use Locale-aware formatters
l10n	Localisation	Adapt software for a specific locale	Translate strings; format currency in INR vs USD
Locale	—	Language + region identifier	en_US, hi_IN, ar_SA, fr_FR
IETF tag	BCP 47	Standard language tag format	en-US, hi-IN, zh-Hans-CN
Collation	—	Locale-aware string sort order	ä sorts after z in Swedish; after a in German
Pluralisation	—	Grammatically correct plural forms	1 item vs 2 items vs 0 items

Java Locale Hierarchy

java.util.Locale – language + country + variant

Locale.ENGLISH – language only (en)

new Locale("en","US") – language + country (en_US)

Locale.forLanguageTag("hi-IN") – IETF BCP 47 (Hindi India)

Locale.forLanguageTag("ar-SA") – Arabic Saudi Arabia (RTL)

ShopVerse supported locales

en-IN – English India (default) INR currency, DD/MM/YYYY dates

hi-IN – Hindi India Devanagari script, INR currency

en-US – English United States USD currency, MM/DD/YYYY dates

ar-SA – Arabic Saudi Arabia (RTL) SAR currency, Hijri calendar

```

@Bean public MessageSource messageSource() {
    ReloadableResourceBundleMessageSource ms =
    new ReloadableResourceBundleMessageSource();
    ms.setBasenames(
        "classpath:i18n/messages",
        "classpath:i18n/errors",
        "classpath:i18n/validation");
    ms.setDefaultEncoding("UTF-8");
    ms.setCacheSeconds(3600); // reload interval in dev; -1 for prod
    ms.setFallbackToSystemLocale(false);
    return ms;
}

@Bean public LocalValidatorFactoryBean validator(MessageSource ms) {
    LocalValidatorFactoryBean b = new LocalValidatorFactoryBean();
    b.setValidationMessageInterpolator(
        new MessageSourceMessageInterpolator(ms));
    return b;
}

```

BaseName	File Pattern	Purpose
i18n/messages	messages.properties, messages_hi_IN.properties	UI properties, UI labels, navigation
i18n/errors	errors.properties, errors_hi_IN.properties	Series of messages returned in API responses
i18n/validation	validation.properties, validation_hi_IN.properties	Bean validation @NotNull, @Size messages

```

@Bean public LocaleResolver localeResolver() {
// Strategy 1: Accept-Language header (REST APIs)
AcceptHeaderLocaleResolver r = new AcceptHeaderLocaleResolver();
r.setSupportedLocales(List.of(
Locale.forLanguageTag("en-IN"),
Locale.forLanguageTag("hi-IN"),
Locale.forLanguageTag("ar-SA")));
r.setDefaultLocale(Locale.forLanguageTag("en-IN"));
return r;
}
// Strategy 2: Session-based (web apps)
SessionLocaleResolver r = new SessionLocaleResolver();
r.setDefaultLocale(Locale.forLanguageTag("en-IN"));
// Strategy 3: URL path variable /api/{locale}/products
// Strategy 4: Cookie
CookieLocaleResolver r = new CookieLocaleResolver("LOCALE");
r.setDefaultLocale(Locale.forLanguageTag("en-IN"));

```

Strategy	Trigger	Best For	ShopVerse
Accept-Language header	HTTP header: Accept-Language	REST APIs; mobile apps	Default for API
Session	Stores locale in HttpSession	Web apps with login	Customer portal
Cookie	Locale stored in browser cookie	Persistent preference	Web storefront
URL path /api/{lang}/	Locale in URL segment	SEO-friendly, shareable	Optimal future

messages.properties (en-IN default)

```
# Navigation
nav.home=Home
nav.products=Products
nav.cart=Cart ({0} items)
# Product page
product.price=Price
product.add_to_cart=Add to Cart
product.out_of_stock=Out of Stock
product.in_stock={0} in stock
# Error messages
error.product.not_found=Product with ID {0} not found
error.insufficient_stock=Only {0} units available; you requested {1}
# Validation
validation.product.name.required=Product name is required
validation.product.price.min=Price must be at least {min}
```

messages_hi_IN.properties (Hindi India)

```
nav.home=■
nav.products=■■■■■■■
product.add_to_cart=■■■■■ ■■■ ■■■■■■
product.out_of_stock=■■■■■ ■■■ ■■■■
error.product.not_found=ID {0} ■■ ■■■■■■ ■■■■ ■■■■
```

MessageSource Usage in Service

```
@Service public class ProductService {
    private final MessageSource ms;
    public String getOutOfStockMessage(int available, int requested, Locale locale) {
        return ms.getMessage("error.insufficient_stock",
            new Object[]{available, requested}, locale);
    }
}
```

```
// Currency formatting - locale-aware
NumberFormat inr = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("hi-IN"));
inr.format(1234567.89); // => "₹12,34,567.89" (Indian grouping)
NumberFormat usd = NumberFormat.getCurrencyInstance(Locale.US);
usd.format(1234567.89); // => "$1,234,567.89" (Western grouping)
// Spring MVC auto-formatting with @NumberFormat
public class ProductDto {
    @NumberFormat(style = Style.CURRENCY) private BigDecimal price;
}

// Date/Time formatting
DateTimeFormatter dtf = DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
    .withLocale(Locale.forLanguageTag("en-IN"));
dtf.format(LocalDate.of(2025,6,17)); // => "17 Jun 2025"
DateTimeFormatter usDtf = DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
    .withLocale(Locale.US);
usDtf.format(LocalDate.of(2025,6,17)); // => "Jun 17, 2025"
```

Locale	Currency Format	Date Format (MEDIUM)	Number Grouping
en-IN (India)	₹12,34,567.89	17 Jun 2025	Indian: 12,34,567
en-US (USA)	\$1,234,567.89	Jun 17, 2025	Western: 1,234,567
hi-IN (Hindi)	₹12,34,567.89	17 1234 2025	Indian: 12,34,567
ar-SA (Arabic)	SAR 1,234,567.89	17 1234 2025	Arabic-Indic numerals

```

@RestControllerAdvice
public class GlobalExceptionHandler {
    private final MessageSource ms;
    @ExceptionHandler(ProductNotFoundException.class)
    public ResponseEntity<ApiError> handle(
        ProductNotFoundException ex, Locale locale) {
        // Locale resolved from Accept-Language header by AcceptHeaderLocaleResolver
        String msg = ms.getMessage("error.product.not_found",
            new Object[]{ex.getProductId()}, locale);
        return ResponseEntity.status(NOT_FOUND)
            .body(new ApiError("PRODUCT_NOT_FOUND", msg, Instant.now()));
    }
}

// ApiError response body
{ "code": "PRODUCT_NOT_FOUND",
  "message": "Product with ID 42 not found", // en-IN
  "message": "ID 42 ■■■■■■■■■■ ■■■■ ■■■■ ■■■■ ■■■■ ■■■■", // hi-IN
  "timestamp": "2025-06-17T10:30:00Z" }

```

Header Sent	Locale Resolved	Error Message Language
Accept-Language: en-IN	en_IN	English (India)
Accept-Language: hi-IN	hi_IN	Hindi
Accept-Language: ar-SA	ar_SA	Arabic (RTL)
No header	en-IN (default)	English (India)

```

<!-- messages.properties key: product.add_to_cart=Add to Cart -->
<button th:text="#{product.add_to_cart}">Add to Cart</button>
<!-- Parameterised message: nav.cart=Cart ({0} items) -->
<span th:text="#{nav.cart(${cartSize})}">Cart</span>
<!-- Locale switcher -->
<a th:href="@{/products(lang=en-IN)}" th:text="#{lang.english}">English</a>
<a th:href="@{/products(lang=hi-IN)}" th:text="#{lang.hindi}">हिन्दी</a>
<!-- dir attribute for RTL languages -->
<html th:attr="dir=${locale.language == 'ar' ? 'rtl' : 'ltr'}">

```

Thymeleaf Expression	Purpose	Example
<code>#{key}</code>	Message lookup – default locale	<code>th:text="#{product.price}" => "Price"</code>
<code>#{key(param)}</code>	Parameterised message	<code>th:text="#{nav.cart(\${n})}" => "Cart (3 items)"</code>
<code>th:attr="dir=..."</code>	Set HTML dir attribute	rtl for Arabic; ltr for Latin scripts
<code>#locale</code>	Access current Locale object	<code>th:if="\${#locale.language == 'hi'}"</code>

Translation Table Pattern

```
-- Products table: language-neutral fields
CREATE TABLE products (
  id BIGINT PRIMARY KEY,
  sku VARCHAR(50) UNIQUE NOT NULL,
  price NUMERIC(12,2) NOT NULL,
  category_id BIGINT REFERENCES categories(id)
);

-- Product translations: one row per locale
CREATE TABLE product_translations (
  id BIGINT PRIMARY KEY,
  product_id BIGINT REFERENCES products(id),
  locale VARCHAR(10) NOT NULL, -- "en-IN", "hi-IN", "ar-SA"
  name VARCHAR(255) NOT NULL,
  description TEXT,
  UNIQUE (product_id, locale)
);

-- Query: get product with localised name for hi-IN
SELECT p.*, pt.name, pt.description
FROM products p
JOIN product_translations pt
ON pt.product_id = p.id AND pt.locale = :locale
WHERE p.id = :productId;
```

Approach	Pros	Cons
Translation table (join)	Clean schema; easy add locale	Join per query; FK integrity
JSONB column per row	No join; single row	JSONB query complex; no FK on translations
Separate table per locale	Simple queries	Table explosion; schema migration nightmare

Aspect	LTR (en, hi)	RTL (ar, he)	Implementation
Text direction	Left to right	Right to left	dir="rtl" on html element
CSS margin/padding	margin-left for indent	margin-right for indent	Use CSS logical props: margin-inline-start
Flexbox direction	row	row-reverse	CSS: direction:rtl flips flex automatically
Number direction	LTR always	LTR always (numbers same)	Numbers stay LTR even in RTL text
Icon position	Icon left of text	Icon right of text	Use CSS logical properties

```

/* CSS: logical properties - work for both LTR and RTL */
.product-card { padding-inline-start: 16px; } /* left in LTR, right in RTL */
.price { margin-inline-end: 8px; } /* right in LTR, left in RTL */
/* Conditional dir via Thymeleaf */
<html xmlns:th=".." th:lang="${#locale.toLanguageTag()}"
th:attr="dir=${#locale.language == 'ar' ? 'rtl' : 'ltr'}">

```

RTL Consideration	ShopVerse Action
Currency symbol position	100 (LTR) vs 100. (RTL – symbol on left in Arabic)
Number formatting	Arabic-Indic digits (٠١٢٣...) for ar-SA if preferred by locale
Product images	Same; images are direction-neutral
Date format	DD/MM/YYYY en-IN; Arabic uses Gregorian or Hijri calendar based on pref

Layer	Configuration	Why
Spring Boot	server.servlet.encoding.charset=UTF-8 (default)	UTF-8 (default in SB)
MySQL/Postgres	charset=UTF8MB4 / client_encoding=Utf8	Get UTF-8 emojis + non-BMP chars
Java source	javac -encoding UTF-8 (default)	Source files with Unicode literals
Properties files	ms.setDefaultEncoding("UTF-8")	ReloadableResourceBundleMessageSource
JSON (Jackson)	UTF-8 default; no BOM	application/json; charset=UTF-8
HTML	<meta charset="UTF-8">	Browser renders UTF-8 content

```
// Unicode Normalisation - important for search and compare
// "é" can be: U+00E9 (single precomposed) or U+0065 + U+0301 (decomposed)
String a = "\u00e9"; // precomposed
String b = "e\u0301"; // decomposed
a.equals(b); // false - different code point sequences
Normalizer.normalize(a, Form.NFC).equals(Normalizer.normalize(b, Form.NFC)); // true
// Postgres: use ICU collation for correct locale-aware sorting
CREATE TABLE products (
  name TEXT COLLATE "hi-IN-x-icu" -- Hindi ICU collation
);
```

```
// Add dependency: com.ibm.icu:icu4j:74.2
// ICU MessageFormat - handles complex plural rules
String pattern =
"{count, plural,
=0 {No items in your cart}
one {You have {count} item in your cart}
other {You have {count} items in your cart}}",
MessageFormat mf = new MessageFormat(pattern, ULocale.forLanguageTag("en-IN"));
mf.format(new Object[]{0, 0}); // => "No items in your cart"
mf.format(new Object[]{1, 1}); // => "You have 1 item in your cart"
mf.format(new Object[]{5, 5}); // => "You have 5 items in your cart"
// Russian has 4 plural forms; standard Java MessageFormat cannot handle this
// ICU4J uses CLDR plural rules → handles all languages correctly
```

Language	Plural Forms	Example
English	2 (one, other)	1 item / 2 items
Hindi	2 (one, other)	1 ■■■■ / 2 ■■■■
Russian	4 (one, few, many, other)	1 ■■■■ / 2 ■■■■■■ / 5 ■■■■■■■■ / 21 ■■■■
Arabic	6 forms	zero/one/two/few/many/other

```

@SpringBootTest @AutoConfigureMockMvc
class I18nIntegrationTest {
    @Autowired MockMvc mvc;
    @ParameterizedTest
    @CsvSource({
        "en-IN, Product with ID 99 not found",
        "hi-IN, ID 99 ₹ 1,234.50"
    })
    void testLocalizedError(String lang, String expectedMsg) throws Exception {
        mvc.perform(get("/api/v1/products/99")
            .header("Accept-Language", lang))
            .andExpect(status().isNotFound())
            .andExpect(jsonPath("$.message").value(expectedMsg));
    }

    @Test void testCurrencyFormatting() {
        NumberFormat inr = NumberFormat.getCurrencyInstance(Locale.forLanguageTag("hi-IN"));
        assertThat(inr.format(new BigDecimal("1234.50")), isEqualTo("₹1,234.50"));
    }
}

```

Test Type	What to Assert
@ParameterizedTest per locale	Error message language matches Accept-Language header
Currency format unit test	NumberFormat.getCurrencyInstance produces correct symbol and grouping
Date format unit test	DateTimeFormatter locale-specific output matches expected pattern
Translation table integration test	Product name returned in correct locale from DB query
RTL detection	dir="rtl" present in HTML response when Accept-Language is ar-SA

■ Do

- ✓ Store all strings in properties files; never hard-code user-visible text in Java or Thymeleaf.
- ✓ Use UTF-8 everywhere: DB charset, HTTP encoding, properties file encoding, JSON.
- ✓ Set `fallbackToSystemLocale=false` on `MessageSource` – unpredictable system locale on servers.
- ✓ Use `Accept-Language` header for REST API locale resolution – stateless, REST-aligned.
- ✓ Apply Unicode normalisation (NFC) before storing and comparing text with diacritics.
- ✓ Use ICU4J for complex plural rules – standard Java `MessageFormat` fails for Russian/Arabic.
- ✓ Use CSS logical properties (`margin-inline-start`) so layout automatically mirrors for RTL.
- ✓ Parameterise i18n tests: run same test for each supported locale to catch missing translations.
- ✓ Use a translation table (`product_translations`) for DB content – scales to any locale without schema change.
- ✓ Cache `MessageSource` with `cacheSeconds=-1` in production; enable reload (`cacheSeconds=5`) in dev.

■ Anti-Patterns

- Hard-coding user-visible strings in Java source – impossible to translate without recompile.
- Using `String.format()` for localised messages – no locale-aware number/date/currency handling.
- Storing translations in a single JSONB column without indexing – unqueryable for search.
- Using UTF-8 in Java but UTF-8MB3 in MySQL – emojis (4-byte) fail to store; use UTF8MB4.
- Ignoring plural forms – "1 items" is grammatically incorrect; use `MessageFormat` or ICU4J.
- Not testing RTL layout – Arabic text in LTR layout breaks the user experience.
- Falling back to system locale on server – production server locale may not be en; unpredictable.

Sub-Chapter	Key Point	One-Liner
12-01 i18n vs l10n	i18n = design; l10n = adapt	Locale = language + country + variant (en-IN, hi-IN)
12-02 MessageBundle	ResourceBundle	setDefaultEncoding UTF-8; fallbackToSystemLocale=false
12-03 Locale Resolver	AcceptHeaderLocaleResolver	Accept-Language: hi-IN → Locale.forLanguageTag("hi-IN")
12-04 Properties	{0} placeholders	ms.getMessage("key", new Object[]{arg}, locale)
12-05 Formatting	NumberFormat.getCurrencyInstance()	Indian grouping: 12,34,567 vs Western 1,234,567
12-06 REST i18n	ExceptionHandler + locale param	locale injected automatically by Spring MVC
12-07 Thymeleaf	{#{key}} syntax	th:text=#{nav.cart(\${n})} → "Cart (3 items)"
12-08 DB i18n	product_translations table	JOIN ON locale; one row per locale per product
12-09 RTL	dir="rtl" + CSS logical properties	margin-inline-start works for both LTR and RTL
12-10 Unicode	NFC normalisation	é (U+00E9) ≠ e+combining-accent without normalise
12-11 ICU4J	CLDR plural rules	Handles Russian 4-form, Arabic 6-form plurals
12-12 Testing	@ParameterizedTest per locale	Assert message language matches Accept-Language header

Interview Quick-Check

Question	Concise Answer
i18n vs l10n?	i18n: engineering for multiple locales (extract strings, use formatters). l10n: adapting for one locale (localization).
How does Spring resolve locale?	AcceptHeaderLocaleResolver reads Accept-Language header. SessionLocaleResolver reads HttpSession attribute.
Why fallbackToSystemLocale=false?	Production server system locale is often en_US or C. Falling back silently shows wrong language; fallbackToSystemLocale=true is for dev.
When to use ICU4J?	When standard Java MessageFormat is insufficient: Russian/Arabic plural rules, number spellout, transliteration.
How to store multilingual DB content?	product_translations table: one row per (product_id, locale). JOIN on locale at query time. Allows any number of locales.